

基于 C++ Easy X 库的图像卷积简单实现

■■■■■■■■中学 ■■■■■级 高■■.■■■班 ■■■■

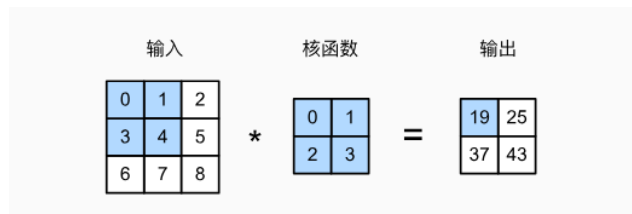
摘要： 图像的卷积在计算机视觉神经网络中有着重要作用，在本文中我们实现了一个简单的基于 C++ 的图像卷积程序

关键词： C++ 卷积 CNN

一、前言

卷积在数学中是通过两个函数 $f(x)$ 和 $g(x)$ 生成第三个函数 $h(x)$ 的一种特殊的积分变换，而在卷积神经网络（CNN）中也有着重要的作用，但由于知识有限且并非必要，本论文先不讨论卷积的数学问题，而是从简单实现原理角度出发，写出一个基础的图像卷积程序。

首先，对于单色图像的卷积，他的步骤如下：第一步，我们需要定义一个卷积核（一般为 3x3），如下图的核函数，第二步，将核函数放至输入图像的左上角，并将核函数上的值与对应的像素值相乘并求和（下图中有四个对应像素），即 $0 * 0 + 1 * 1 + 3 * 2 + 4 * 3 = 19$ ，第三步，将卷积核向右逐行扫描，并循环第一二步，直到遍历完整个输入图像，卷积核的求和结果组成输出图像。



简单卷积示意图

但是有一个明显的问题，即输出相较于输入小了一圈，因此我们应该在输入图片边“围上”一圈像素值为 0 的像素，以确保输出和输入大小相同。

接下来，让我们尝试用 C++ 的 Easy X 库进行卷积操作的简单实现。

二、代码实现

1、卷积核及输入图像的读取

首先，定义一个 3x3 的卷积核以及 200x200 的输入和输出（留出 2 行空白），并读取卷积核和输入函数（由于 Easy X 图像储存为一维数组，为方便编程，这里选择将其转化为二维数组，代码较长）

```
1. int Input[202][202] = { 0 }; //输入
2. float Output[200][200] = { 0 }; //输出
3. float Convolution[3][3] = { 0 }; //卷积核
4. int OutputX=200, OutputY=200; //(W-F+2P/S)+1 源x/y-卷积核x/y+边缘填充*2 ÷ 步长 +1
5. int main(){
6.     IMAGE ph;
7.     loadimage(&ph, _T(".\\photo.png")); //输入卷积核内容
8.     cout << "卷积核内容(3x3(单色通道)):" << endl;
9.     for (int x = 0; x < 3; x++)
10.        for (int y = 0; y < 3; y++)
11.            cin >> Convolution[x][y]; //读取源图像内容
12.     initgraph(800,800); //创建窗口
13.     setbkcolor(WHITE); //白色背景
14.     clearcliprgn(); //刷新背景
```

```

15. putimage(51, 51, &ph); //显示图片
16. for (int x = 51; x < 251; x++) //读取像素值，由于黑白照片 RGB 单通道值相等，所以这里取 R 通道值
17.     for (int y = 51; y < 251; y++){
18.         Input[x - 50][y - 50] = GetRValue(getpixel(x, y)); //R
19.     }
20. }

```

2.进行卷积

首先将卷积核的中心移至输入的(1,1)和输出的(0,0)，并逐行进行卷积，这里直接将卷积结果写入输出层

```

1. //开始卷积
2. for (int x = 0; x < OutputX; x++)
3.     for (int y = 0; y < OutputY; y++){
4.         for (int i = 0; i < 3; i++)
5.             for (int j = 0; j < 3; j++){
6.                 Output[x][y] += Convolution[i][j] * Input[x + i][y + j];

```

3.绘制图像

```

1. for (int x = 250; x < 450; x++)
2.     for (int y = 500; y < 700; y++){
3.         putpixel(x, y, RGB(Output[x - 250][y - 500], Output[x - 250][y - 500], Output[x - 250][y - 500]));

```

完整代码（需安装 Easy X 库）：

```

1. #include<iostream>
2. #include<cmath>
3. #include<conio.h>
4. #include<easyx.h>
5. using namespace std;
6.
7. int Input[202][202] = { 0 }; //源图像 RGB
8. float Output[200][200] = { 0 }; //池化层后
9. float Convolution[3][3] = { 0 }; //卷积核
10. int OutputX = 200, OutputY = 200; //(W-F+2P/S)+1 源x/y-卷积核x/y+边缘填充*2 ÷ 步长 +1
11. int main() {
12.     IMAGE ph;
13.     loadimage(&ph, _T(".\\photo.png")); //输入卷积核内容
14.     cout<< "卷积核内容(3x3(单色通道)):" <<endl;
15.     for (int x = 0; x < 3; x++)
16.         for (int y = 0; y < 3; y++){
17.             cin >> Convolution[x][y]; //读取源图像内容
18.         }
19.     initgraph(800, 800); //创建窗口
20.     setbkcolor(WHITE); //白色背景
21.     clearcliprgn(); //刷新背景
22.     putimage(51, 51, &ph); //显示图片

```

```

22. for (int x = 51; x < 251; x++) //读取像素值，由于黑白照片RGB 单通道值相等，所以这里取R 通
    道值
23.     for (int y = 51; y < 251; y++) {
24.         Input[x - 50][y - 50] = GetRValue(getpixel(x, y)); //R
25.     }
26. //开始卷积
27. for (int x = 0; x < OutputX; x++)
28.     for (int y = 0; y < OutputY; y++)
29.         for (int i = 0; i < 3; i++)
30.             for (int j = 0; j < 3; j++)
31.                 Output[x][y] += Convolution[i][j] * Input[x + i][y + j];
32. for (int x = 250; x < 450; x++)
33.     for (int y = 500; y < 700; y++)
34.         putpixel(x, y, RGB(Output[x - 250][y - 500], Output[x - 250][y - 500], Output[x - 2
            50][y - 500]));
35. _getch();
36. closegraph();
37. return 0;
38. }

```

三、不同卷积核的尝试

1.均值滤波：

```

0.11 0.11 0.11
0.11 0.11 0.11
0.11 0.11 0.11

```



2.高斯滤波

```

0.0625 0.125 0.0625
0.125 0.25 0.125
0.0625 0.125 0.0625

```



3.锐化卷积核

```

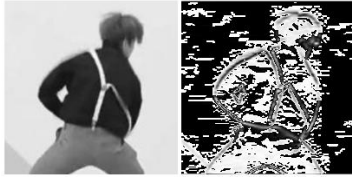
-1 -1 -1
-1 9 -1
-1 -1 -1

```



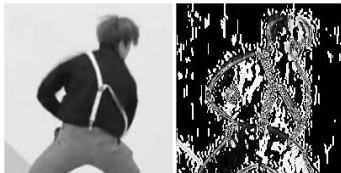
4. Prewitt 算子（左右边缘检测）

```
-1 0 1  
-1 0 1  
-1 0 1
```



5. Sobel 算子

```
1 2 1  
0 0 0  
-1 -2 -1
```



四、结语

通过上面的简单实现，我们可以明显地感受到图像卷积的重要性，我也希望有朝一日可以把我的程序改进为卷积神经网络的池化层。

参考文献

- [1] [6.1. 从全连接层到卷积 — 动手学深度学习 2.0.0 documentation \(d2l.ai\)](#)
- [2] [常用图像卷积核类型小结 - 知乎 \(zhihu.com\)](#)